Single Element in a Sorted Array

Difficulty: Medium
Patterns: Array, Binary Search, Pair Parity
LeetCode: https://leetcode.com/problems/single-element-in-a-sorted-array/description/

1. Problem Summary

We are given a sorted array where every value appears exactly twice, except one value that appears once. Return that single value.

The solution must run in `O(log n)` time and use `O(1)` extra space, so a linear scan or a hash map is not the intended final answer.

2. Constraints and Observations

- `1 <= nums.length <= 10^5`
- `0 <= nums[i] <= 10^5`
- The array is sorted.
- Every element appears twice except one.
- The required time is `O(log n)`, which strongly points to binary search.
- The array length is always odd because it contains pairs plus one single element.

The sorted order means equal values are adjacent. Before the single element, pairs begin at even indices. After the single element, that pairing pattern shifts, so pairs begin at odd indices.

3. Pattern Recognition

This is not a normal binary search for a known target. It is a binary search over a broken pattern.

The pattern is:

- Before the single element: `nums[even] == nums[even + 1]`
- At or after the single element: this even-index pair pattern no longer holds

So each binary-search step asks: "Is the left side still perfectly paired, or has the single element already appeared?"

4. Naive Approach

The first natural idea is to scan the array and compare neighbors.

1. Move from left to right.
2. If `nums[i] != nums[i + 1]`, then `nums[i]` is the single value.
3. Otherwise skip the pair by moving `i += 2`.

This uses `O(1)` space, but it can inspect almost the entire array, so the time is `O(n)`. That violates the required `O(log n)` time.

Another possible idea is XOR, because pairs cancel out. That also finds the answer in `O(n)` time and `O(1)` space, but it ignores the sorted-array structure and still fails the time requirement.

5. Key Intuition

Pairs are predictable until the single element interrupts them.

```
nums:   [1, 1, 2, 3, 3, 4, 4, 8, 8]
index:   0  1  2  3  4  5  6  7  8
```

Before `2`, the pair `(1, 1)` starts at even index `0`.

After `2`, the pair `(3, 3)` starts at odd index `3`, then `(4, 4)` starts at odd index `5`, and so on.

So the single element is exactly where the pair-start parity changes.

6. Optimal Solutions Ranked

Variant 1: Neighbor-aware binary search

- Idea: Choose a middle index, adjust it to be even, then compare `nums[mid]` with `nums[mid + 1]`.
- Time: `O(log n)`
- Space: `O(1)`

- Tradeoff: Very readable and interview-friendly. This is the recommended implementation.

Variant 2: `mid ^ 1` binary search

- Idea: Compare each `mid` with its pair partner using `mid ^ 1`. For an even index, `mid ^ 1` is `mid + 1`; for an odd index, it is `mid - 1`.
- Time: `O(log n)`
- Space: `O(1)`
- Tradeoff: Shorter code, but the bit trick can be less obvious in an interview unless explained clearly.

No better asymptotic bound is possible under the comparison model because the required answer depends on locating a break among index ranges. Binary search reaches the required `O(log n)` time, and `O(1)` space is already minimal.

7. Most Optimal Approach

Use binary search on even pair starts.

1. Pick `mid`.
2. If `mid` is odd, move it left by one so it becomes the first index of a possible pair.
3. Compare `nums[mid]` and `nums[mid + 1]`.
4. If they are equal, the left side up to that pair is valid, so the single element must be to the right.
5. If they are not equal, the pairing has already broken, so the single element is at `mid` or to the left.

The search converges to the single element.

8. Algorithm

1. Set `left = 0` and `right = len(nums) - 1`.
2. While `left < right`, compute `mid = (left + right) // 2`.
3. If `mid` is odd, decrement it by `1` so that it points to an even index.
4. If `nums[mid] == nums[mid + 1]`, move right with `left = mid + 2`.
5. Otherwise, move left with `right = mid`.
6. Return `nums[left]`.

9. Visual Explanation

The important signal is the shift in pair-start parity. For `nums = [1,1,2,3,3,4,4,8,8]`, the value `2` breaks the pair alignment.

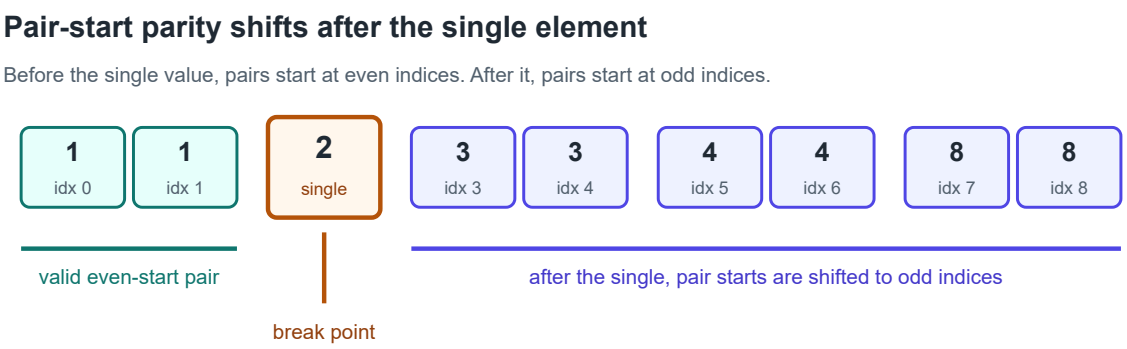

Diagram: the single value causes the duplicate-pair alignment to flip.

10. Dry Run

Example: `nums = [3,3,7,7,10,11,11]`

Step	left	right	raw mid	adjusted mid	Comparison	Decision
1	0	6	3	2	`nums[2] == nums[3]`, so `7 == 7`	Left side is paired. Move `left = 4`.
2	4	6	5	4	`nums[4] == nums[5]`, so `10 != 11`	Break is at `mid` or left. Move `right = 4`.
End	4	4	-	-	Search converged	Return `nums[4] = 10`.

11. Correctness Argument

Invariant: At the start of every loop, the single element is inside the search interval `[left, right]`.

When `mid` is adjusted to be even, it represents a possible pair start.

- If `nums[mid] == nums[mid + 1]`, then the pair beginning at `mid` is valid. Since all values before the single element follow the even-start pairing pattern, the single element cannot be at or

before `mid + 1`. Moving `left` to `mid + 2` preserves the invariant.

- If `nums[mid] != nums[mid + 1]`, the even-start pairing pattern has broken at `mid`. The single element is either exactly `mid` or somewhere to the left. Moving `right` to `mid` preserves the invariant.

Each iteration strictly shrinks the interval, so the loop terminates. When `left == right`, the invariant says the only remaining index contains the single element. Therefore, returning `nums[left]` is correct.

12. Complexity Analysis

- Time: `O(log n)`, because each iteration discards about half of the remaining search range.
- Space: `O(1)`, because only a few index variables are used.

13. Edge Cases

- Single element array, such as `[5]`: the loop never runs, and `5` is returned.
- Single element at the beginning, such as `[2,3,3,4,4]`: the first failed pair comparison keeps moving left.
- Single element at the end, such as `[1,1,2,2,9]`: valid pairs keep pushing the search right.
- Smallest non-trivial layout, such as `[1,2,2]` or `[1,1,2]`: the adjusted-mid comparison still works.

14. Final Clean Code

```
from typing import List


class Solution:
    def singleNonDuplicate(self, nums: List[int]) -> int:
        left, right = 0, len(nums) - 1

        while left < right:
            mid = (left + right) // 2

            if mid % 2 == 1:
                mid -= 1

            if nums[mid] == nums[mid + 1]:
                left = mid + 2
            else:
                right = mid

        return nums[left]
```

15. Key Takeaways

> - Main pattern: binary search over a structural break, not over a target value.
> - Main trick: before the single element, duplicate pairs start at even indices; after it, they start at odd indices.
> - Interview explanation: "I force `mid` to an even index, test whether the pair at `mid, mid + 1` is valid, and discard the half that cannot contain the broken pair pattern."